CS-1101
PROGRAMMING FUNDAMENTALS

Lecture 03

By

Engr. Muhammad Awais

November 4, 2024

Lecture Contents

- Variables
- Constants
- Comments
- Special Symbols
- Reserved Keywords
- Identifiers
- Datatypes

Lecture Objectives

- Learn how to declare and initialize variables to store different types of data.
- Explore various data types and understand their roles in representing information.
- Identify identifiers and their importance in naming program elements.
- Recognize the use of constants for fixed values that do not change during program execution.

Variables

- A variable in programming is a named storage location in memory that holds a value that can be changed during program execution.
- Variable Declaration

```
dataType identifier, identifier, . . .;
```

```
int feet;
```

- Variable Initialization
- There are two ways to initialize a variable:
  - By using the assignment statement

    ```
    feet = 35;
    ```

  - By using a read statement

    ```
    cin >> feet;
    ```

Variables Cont…

- Consider various ways of declaring variables:
  - Method 1

```
int feet;
int inch;
double x;
double y;
```

  - Method 2

```
int feet, inch;
double x, y;
```

  - Method 3

```
int a,b;double x,y;
```

- All are correct; however, the third is hard to read.

Variables Cont…

- Variables can be initialized when declared:

```
int first=13, second=10;
char ch=' ';
double x=12.6;
```

- All variables must be initialized before they are used
  - But not necessarily during declaration

Constants

- A constant in programming is a value that, once defined, cannot be changed during the execution of the program.
- Constants are used to represent fixed values that remain the same throughout the program and preventing accidental modification of important values.
- Syntax to create constant in C++.

```
const dataType identifier = value;
```

Constants Cont…

- Syntax to create constant in C++.

```
const dataType identifier = value;
```

- In C++, `const` is a reserved word.
- Constant must be initialized at the time of declaration

EXAMPLE 2-11

Consider the following C++ statements:

```
const double CONVERSION = 2.54;
const int NO_OF_STUDENTS = 20;
const char BLANK = ' ';
const double PAY_RATE = 15.75;
```

Constants Cont…

- Another way of creating a constant in C++ is using
  - #define preprocessor directives.

```
#define PI 3.14159
```

Assignment Statement

- The assignment statement takes the form:

```
variable = expression;
```

- Expression is evaluated and its value is assigned to the variable on the left side
- In C++, = is called the assignment operator

Assignment Statement Cont…

EXAMPLE 2-13

```
int num1, num2;
double sale;
char first;
string str;

num1 = 4;
num2 = 4 * 5 - 11;
sale = 0.02 * 1000;
first = 'D';
str = "It is a sunny day.";
```

EXAMPLE 2-14

```
1.  num1 = 18;
2.  num1 = num1 + 27;
3.  num2 = num1;
4.  num3 = num2 / 5;
5.  num3 = num3 / 4;
```

Comments

- Comments are for the reader, not the compiler
- Two types:
  - Single line

```
// This is a C++ program. It prints the sentence:
// Welcome to C++ Programming.
```

  - Multiple line

```
/*
   You can include comments that can
   occupy several lines.
*/
```

Comments

- A well-documented program is easier to understand and modify
- You use comments to document programs
- Comments should appear in a program to:
  - Explain the purpose of the program
  - Identify who wrote it
  - Explain the purpose of particular statements

Special Symbols

- Special symbols in C++ are characters or symbols with predefined meanings in the language that are used to perform specific operations or represent various programming constructs.
- Examples of special symbols

```
+ - * / . ; {} ? , <= != == >= ()
```

Reserved Words (Keywords)

- Reserved words, keywords,
- Keywords (also known as reserved words) have special meaning to the C++ compiler and are always written or typed in short(lower) cases.
- It can't be used for a variable name or function name.

Reserved Words (Keywords)

- Some keywords are:
  - `int`
  - `float`
  - `double`
  - `char`
  - `const`
  - `void`
  - `return`

Identifiers

- Any name given to the part of program is known as identifier.
  - Variable Name or Function Name or Class Name

<u>Rules for Identifiers</u>

- Identifiers can only contain letters (a-z, A-Z), digits (0-9), and underscores (_).
- They cannot contain spaces or special characters (like @, #, $, etc.).
- An identifier must begin with a letter or an underscore. It cannot start with a digit.
- Identifiers cannot be C++ keywords (such as int, for, while, class, etc.).Using keywords as identifiers will cause compilation errors.
- Identifiers are case-sensitive in C++. For example, Value and value are considered two different identifiers.

Identifiers (continued)

- The following are legal identifiers in C++:
  - `first`
  - `conversion`
  - `payrate`
- The following are illegal identifiers in C++:

TABLE 2-1 Examples of Illegal Identifiers

Illegal Identifier	Description
`employee Salary`	There can be no space between `employee` and `Salary`.
`Hello!`	The exclamation mark cannot be used in an identifier.
`one+two`	The symbol + cannot be used in an identifier.
`2nd`	An identifier cannot begin with a digit.

Identifiers (continued)

Identifiers	Valid / Invalid
23age	Invalid
Father name	Invalid
age@	Invalid
While	Valid
_name	Valid
if	Invalid
firstname	Valid
First_name	Valid
Firstname123	Valid

Naming Identifiers

- Identifiers can be self-documenting:
  - `CENTIMETERS_PER_INCH`
- Avoid run-together words :
  - `annualsale`
  - Solution:
    - Capitalize the beginning of each new word
      - `annualSale`
    - Inserting an underscore just before a new word
      - `annual_sale`

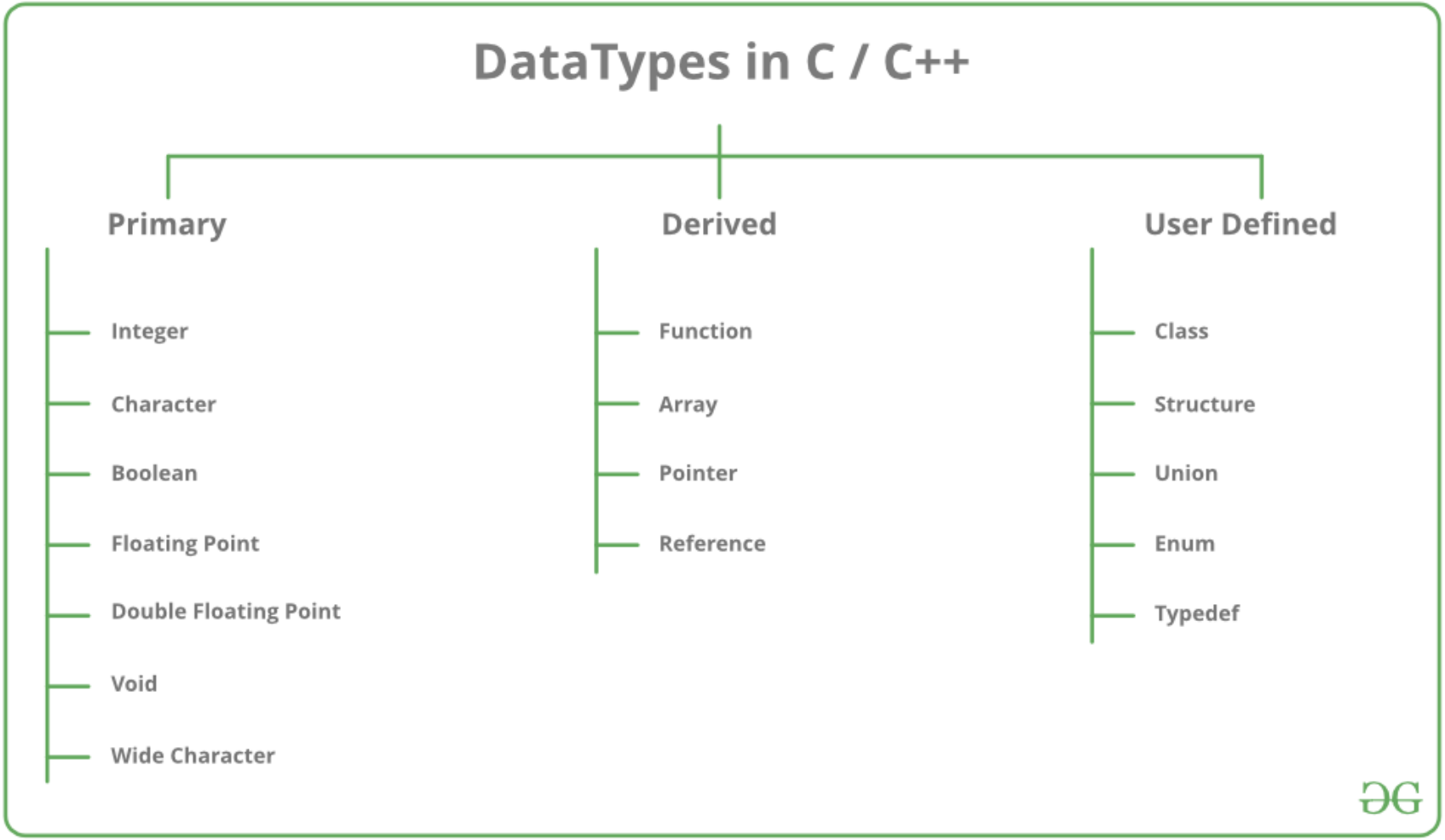
Naming conventions

```
camelCase
snake_case
PascalCase
```

Data Type

- Data types define the type and size of data that can be stored in a variable.
- Choosing the right data type optimizes memory usage and ensures proper handling of data.

Categories of Datatypes



`int` – Primary Datatype

- Examples:

  `-6728`

  `0`

  `78`

  `+763`
- Positive integers do not need a + sign
- Size of int = 4 bytes
- Size of primary datatype be calculated using sizeof()
- Ranges : -2147483648 to 2147483647

`bool` – Primary Datatype

- `bool` type
  - Two values: `true` and `false`
  - Manipulate logical (Boolean) expressions
- `true` and `false` are called logical values
- `bool`, `true`, and `false` are reserved words.
- Size of bool = 1 bytes

`char` – Primary Datatype

- Used for <u>characters</u>: letters, digits, and special symbols
- Each character is enclosed in single quotes
  - `'A','a','0','*','+','$','&'`
- A blank space is a character and is written

`'  '`, with a space left between the single quotes.

- Size of char = 1 byte

Floating-Point – Primary Datatype

- In C++, a **floating-point value** refers to a data type that is used to represent real numbers (numbers that can have fractional parts).
- In C++, there are primarily two standard floating-point data types:
  - float
  - double
- <u>Memory</u>:
  - float : 4 bytes.
  - double : 8 bytes

Floating-Point – Primary Datatype

- Precision: maximum number of significant digits
  - float values are called single precision
    - Maximum number of significant digits (decimal places) for float values is 6 or 7
  - double values are called double precision
    - Maximum number of significant digits for double is 15

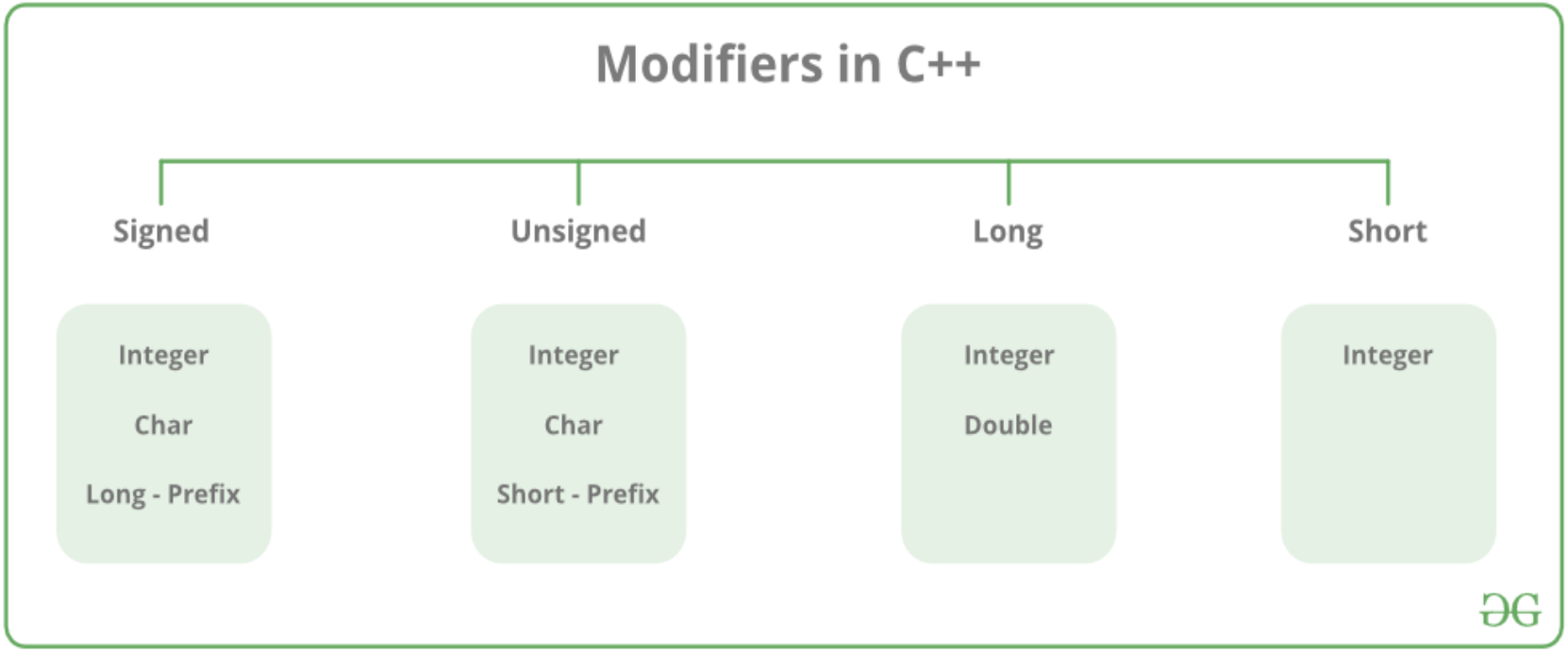
Primary Datatype – Sizes in bytes

Datatype	Size in bytes
int	4
float	4
double	8
char	1
bool	1
void	0

sizeof(datatype) can be used to find the memory occupied by a variable of a particular datatype.

Datatype Modifiers

- Datatype modifiers are used with built-in data types to modify the length of data that a particular data type can hold.



EXERCISE

- Write a C++ program that:
  - Initializes two variables, a and b, with any integer values of your choice.
  - Performs the following operations on these variables:
    Addition (+) Subtraction (-) Multiplication (*) Division (/).
  - Displays the result of each operation with a clear label.

TASK 1 – CALCULATING DISTANCE

- Write a C++ program to calculate the squared distance between two points in a 2D plane using initialized values for coordinates.

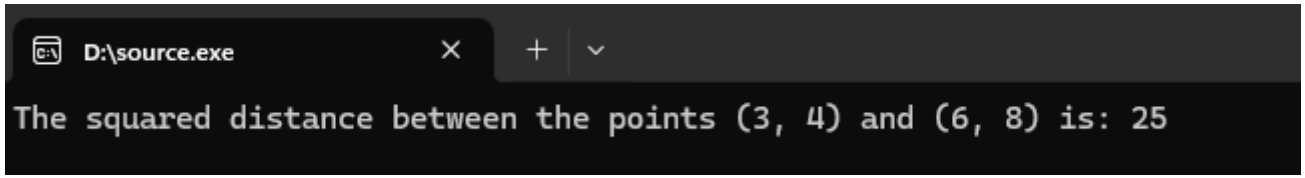
  This exercise reinforces variable declaration and initialization in C++.

Instructions:

- Initialize the coordinates $x1, y1$ and $x2, y2$ directly in the code as double variables.
- Use the formula:

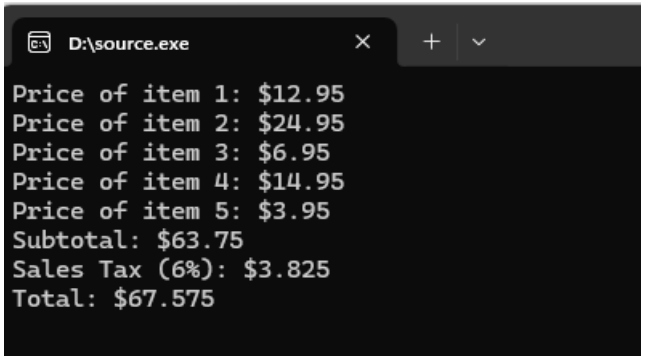
$$\text{squared_distance} = (x_2 - x_1)^2 + (y_2 - y_1)^2$$

Display the squared distance in the specified format.



TASK 2 : TOTAL PURCHASE CALCULATION

- A customer is buying five items in a store. The prices of these items are as follows:
  - Item 1: $12.95
  - Item 2: $24.95
  - Item 3: $6.95
  - Item 4: $14.95
  - Item 5: $3.95



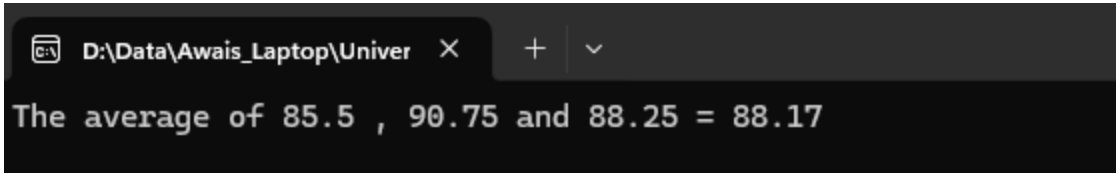
Write a C++ program that stores the prices of these five items in separate variables. Your program should:

- Display the price of each item individually.
- Calculate and display the subtotal of the sale (sum of all items).
- Calculate and display the sales tax (6% of the subtotal).
- Display the total amount (subtotal plus sales tax).

Note: Use constants where appropriate (e.g., for the sales tax rate). Use appropriate datatypes to store result and values in variables & constants. This exercise focuses on understanding variables ,constants and datatypes.

TASK 3 : Average of Predefined Test Scores Rounded to Two Decimal Places

- Write a C++ program that calculates the average of three predefined test scores and displays the result rounded to two decimal places. The program should have the test scores initialized in the code and, after calculating the average, format the output to show exactly two decimal places for clarity.



TASK 1 : Arithmetic Operations on User-Entered Integers Using cin

- Write a C++ program that prompts the user to enter two integer numbers. The program should then use these numbers to perform and display the results of all basic arithmetic operations: addition, subtraction, multiplication, integer division, and modulus

```
D:\Data\Awais_Laptop\Univer

Enter two numbers :
Number 1 :11
Number 2 : 2

        Addition:
                11 + 2 = 13
        Subtraction:
                11 - 2 = 9
        Multiplication:
                11 x 2 = 22
        Division:
                11 / 2 = 5
        Moduls:
                11 % 2 = 1
```

TASK 2 : BMI Calculator

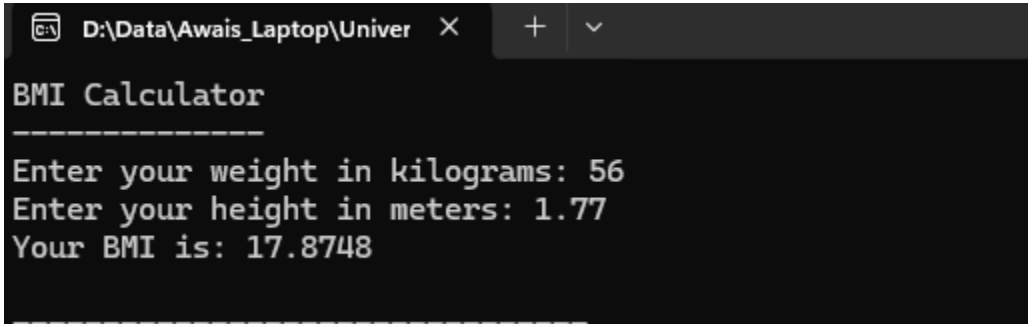
- Develop a C++ program to calculate a person's Body Mass Index (BMI) based on their weight (in kilograms) and height (in meters). The program should prompt the user to input their weight and height, then calculate and display their BMI using the formula:

BMI Formula:

$$BMI = Weight(Kg) / (Height(m))^2$$

Where:

- Weight is in kilograms (kg)
- Height is in meters (m)



TASK 3 : Trip Cost Calculator

- Write a C++ program to estimate the total cost of a trip based on distance, vehicle fuel efficiency, and fuel cost per liter. The program should prompt the user for these values, calculate the total cost of fuel for the trip, and display the result using the following formula::

Trip Cost Formula:

$$\text{Total Trip Cost} = \left(\frac{\text{Distance (km)}}{\text{Fuel Efficiency (km per liter)}}\right) \times \text{Fuel Cost per Liter}$$

- Distance is the total trip distance in kilometers (km).
- Fuel Efficiency is the vehicle's fuel efficiency in kilometers per liter (km/l).
- Fuel Cost per Liter is the cost of fuel per liter in the PKR

```
D:\Data\Awais_Laptop\Univer
Trip Cost Estimator
-------------------
Enter distance of the trip in kilometers: 150
Enter vehicle's fuel efficiency (km per liter): 10
Enter fuel cost per liter: 250
Total trip cost: Rs 3750
```

REFERENCES

- https://www.geeksforgeeks.org/cpp-data-types/